

AI-Based Road Condition Detection of Potholes and Speed Breakers for Autonomous Vehicles Using YOLOv11

Meet Shroff, Aditya Shah, Kaushal Rathod and Drashtiben Vaghasiya
W Booth School of Engineering Practice and Technology, McMaster University
Hamilton, Ontario, Canada

Abstract—This paper presents a real-time road-hazard detection system aimed at enhancing the safety and navigation capabilities of autonomous vehicles. Leveraging the YOLOv11 Large object-detection model, the system is trained to identify and classify two key road anomalies: potholes and unmarked speed breakers. A custom dataset of more than 1,500 annotated images was curated using Roboflow, combining varied lighting, weather, and road conditions to ensure robust performance. The model was trained and evaluated using the Ultralytics framework on Google Colab, achieving a mean average precision (mAP@50–95) of 0.49, with strong results on both image- and video-based inference. Extensive data augmentation and architectural optimizations were applied to improve generalization and real-time speed. The trained model reliably detects multiple hazards per frame in urban driving videos and still images, and its architecture supports deployment on edge devices for integration into autonomous-vehicle perception pipelines. While performance is strong under standard conditions, challenges remain in extreme weather, low-light environments, and certain false positives such as manhole covers. Future work targets dataset expansion, GPS integration, transformer-based backbones, lane filtering, and model light-weighting for resource-constrained hardware.

Keywords—road hazard detection, YOLOv11, object detection, autonomous vehicles, potholes, speed breakers, deep learning, real-time inference

I. INTRODUCTION

Maintaining road quality is essential for ensuring safety, comfort, and efficiency in modern transportation systems, especially with the rise of autonomous vehicles (AVs). Potholes, broken roads, and unmarked speed breakers not only threaten vehicle integrity and passenger safety but also challenge the perception systems of autonomous navigation. This work presents a real-time road-hazard detection solution that uses a YOLOv11 Large object-detection model to identify and classify such anomalies from video input. By integrating this system into the perception pipeline of AVs, the project aims to enhance hazard awareness, enable smarter driving decisions, and support infrastructure maintenance through accurate, real-time road-condition monitoring.

A. Problem Statement

Poor road infrastructure presents a significant challenge to modern transportation. These hazards are a leading cause of vehicle damage, increased maintenance costs, and passenger discomfort, and they pose safety risks to both human drivers and AVs. For AVs, which rely heavily on accurate environmental

perception, undetected road anomalies can cause sudden braking, unsafe lane changes, or delayed responses, endangering passengers and surrounding traffic. Despite advances in object detection and path planning, many AV systems still struggle to recognize subtle or irregular surface defects in real-world conditions, particularly when lighting, weather, or road markings are poor. Traditional sensors such as LiDAR and radar may overlook or misclassify such hazards without visual cues. There is therefore a pressing need for a robust, real-time, vision-based detection system that supports safer decision-making in autonomous driving.

B. Objective

The objective of this project is to design and implement a real-time road-hazard detection system tailored for autonomous-vehicle environments, using YOLOv11 to detect and classify potholes, broken surfaces, and speed breakers. The system operates on video input, processes frames efficiently, and outputs annotated visual data suitable for integration into the vehicle's perception and control pipeline. By recognizing hazards in advance, the system enables proactive adjustments in speed, suspension dynamics, and steering, improving ride safety, passenger comfort, and vehicle longevity. Detection results are also structured to support GPS tagging and temporal mapping, enabling real-time road-quality databases for fleet operators, urban planners, and navigation systems.

C. Scope

This project focuses on an automated deep-learning system that detects potholes and speed breakers from road images and videos using a real-time object-detection model. The scope covers:

- **Data acquisition and annotation:** curating a diverse dataset of road images and annotating it to train a supervised detector.
- **Model development and training:** building a YOLOv11 detection pipeline with systematic tuning of hyperparameters and augmentations.
- **Performance evaluation:** using precision, recall, mAP@0.5 and mAP@0.5:0.95, with per-class analysis and training-curve interpretation.
- **Inference benchmarking:** measuring inference speed (FPS) on CPU and GPU platforms to assess real-time suitability.

- **Real-time testing:** validating the model on video streams and standalone images under varying lighting and road conditions.
- **Hardware-deployment feasibility:** assessing compatibility with edge devices such as Jetson Nano, Jetson Xavier, and Raspberry Pi 4, and exploring quantization and TensorRT conversion.

The scope is limited to two anomaly classes—potholes and speed breakers. Severity grading, segmentation, and automated vehicle-control integration are beyond the current scope; real-time feasibility is evaluated but full embedded deployment is not performed.

II. LITERATURE REVIEW

A. Overview of Object-Detection Techniques

Object detection is a fundamental computer-vision task involving the identification and localization of objects within an image. It underpins applications such as autonomous driving, surveillance, medical imaging, and industrial inspection. Early methods relied on handcrafted features such as Haar cascades (Viola and Jones, 2001), histograms of oriented gradients (Dalal and Triggs, 2005), and deformable part-based models. These approaches required domain-specific feature engineering and performed poorly under variable lighting, cluttered backgrounds, and complex shapes.

B. Use of Deep Learning

With the advent of deep learning, object detection was transformed. Deep convolutional neural networks (CNNs) learn hierarchical features directly from data, eliminating manual feature design. Modern detectors fall into two categories: two-stage detectors such as R-CNN, Fast R-CNN, and Faster R-CNN, which first generate region proposals and then classify and refine bounding boxes for high accuracy; and single-stage detectors such as YOLO (You Only Look Once) and SSD, which predict bounding boxes and class probabilities in a single forward pass, making them fast and suitable for real-time applications.

C. Previous Work on Pothole and Speed-Breaker Detection

Road-surface monitoring has long been a key research area in intelligent transportation systems. Early image-based work used edge detection, contour analysis, and colour thresholding. Koch and Brilakis (2011) proposed edge-based segmentation combined with texture analysis for pothole detection from grayscale images; the method was effective under controlled lighting but lacked robustness to varying weather. Zhang et al. (2014) used 2-D texture features and shape descriptors, but generalization across surfaces was poor.

With the rise of CNNs, learning-based methods began to outperform classical approaches. Maeda et al. (2018) introduced a road-damage dataset and used Faster R-CNN to detect cracks and potholes. Saini et al. (2020) implemented YOLOv3 for real-time pothole detection but struggled with false positives from road textures and shadows. Kumar et al. (2021) used SSD-MobileNet to detect speed breakers and potholes from dashcam footage for lightweight deployment. Patil et al. (2021) deployed YOLO on Jetson Nano using quantization and pruning, and Rai

et al. (2022) proposed a GPS-integrated YOLOv5 system that mapped potholes for maintenance planning. These works illustrate a growing trend toward real-time, embedded-friendly, and geospatially aware detection pipelines.

D. Limitations in Existing Approaches

Despite these advances, existing approaches face several limitations that hinder real-world deployment. A key challenge is the lack of generalization across diverse environmental conditions: models that perform well on specific datasets often degrade under different lighting, weather, and road textures. A second limitation is the high rate of false positives caused by visual similarity between anomalies and non-defective regions such as shadows, oil stains, puddles, or faded markings. Models also underperform on small or partially occluded objects, especially at frame edges or under motion blur. In video applications, the absence of temporal consistency causes detections to flicker between frames because most detectors operate independently per frame. Finally, many methods focus solely on detection without severity grading, GPS integration, or actionable reporting, limiting their value for civic infrastructure systems.

III. DATASET

The effectiveness of any deep-learning detector strongly depends on the quality and diversity of its training data. A custom dataset was created to capture real-world road conditions, focusing on two anomaly types: potholes and speed breakers. A significant portion of the data was sourced from Roboflow Universe, an open repository of high-quality annotated datasets. Roboflow’s integrated annotation and export tools accelerated preprocessing and enabled seamless conversion to YOLO-compatible formats.

The custom dataset was built by merging three independently annotated datasets: one pothole-focused dataset and two speed-breaker datasets, providing a rich variety of road conditions under different lighting, weather, and environments.

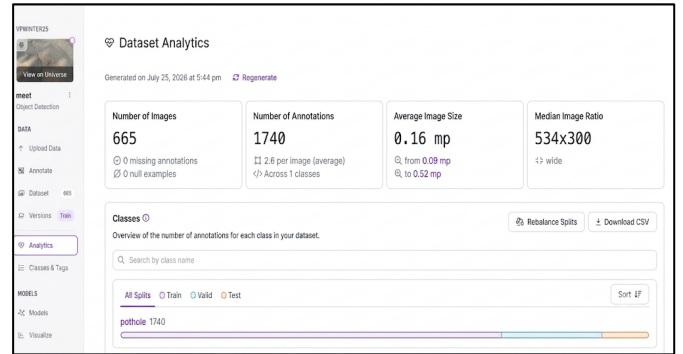


Fig. 1. Sample images from the pothole detection dataset.

The first dataset focused exclusively on pothole detection. It contained 665 images and 1,740 annotations, capturing potholes from various angles and lighting conditions. Most images came from urban road scenarios, including uneven surfaces, water-logged potholes, and partially occluded anomalies, and formed a strong base for the pothole class.

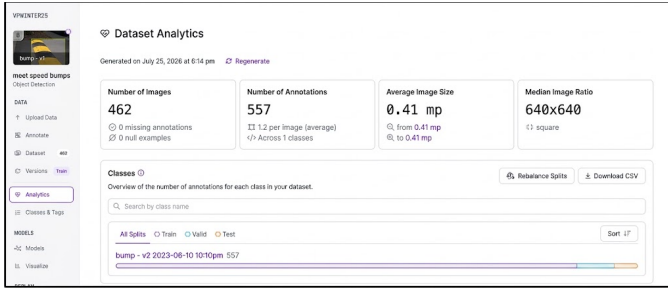


Fig. 2. Sample images from the first speed-breaker dataset.

The second dataset consisted of 462 images with 557 speed-breaker annotations, primarily from road surveillance or dashcam footage of visible, well-marked speed bumps under daylight conditions. Some class-labelling inconsistencies were later resolved during merging.

The third dataset also targeted speed breakers and contained 457 images with a broader variety of conditions, including night-time captures and faded, unmarked, or shadowed bumps. It improved the model’s ability to detect less-visible speed breakers, though verbose labels such as “bump - v2 2023-06-10 10:10pm” were normalized during cleaning.

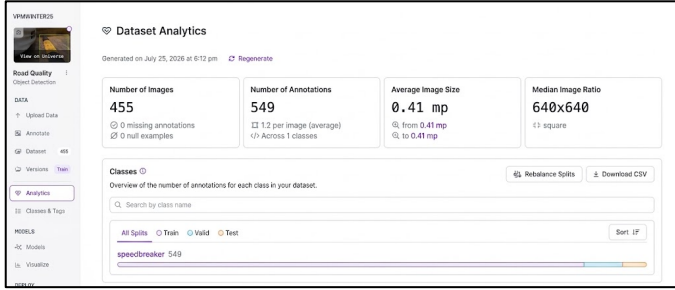


Fig. 3. Sample images from the second speed-breaker dataset.

Using Roboflow, the three datasets were merged into a project titled “New_merged” containing 1,584 images and 2,846 bounding-box annotations. Label inconsistencies were resolved by consolidating all speed-breaker labels under a single class, speedbreaker, ensuring uniformity and reducing class ambiguity. The merge also verified annotation formats, aligned bounding-box definitions to YOLO standards, and reviewed metadata for corrupted or null entries; only two null or duplicate images were found and removed.

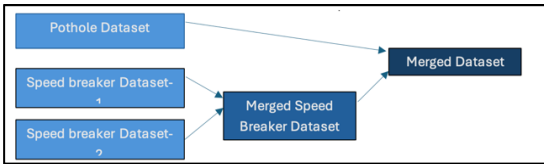


Fig. 4. Steps for merging the dataset using Roboflow

To ensure consistency, all images were resized to 640×640 pixels, the expected input size for YOLOv11, unifying varying aspect ratios and resolutions. A manual review corrected or removed mislabelled images and normalized class names. The dataset was then split into training (80%), validation (10%), and testing (10%) subsets with balanced class distribution. The final dataset contains 1,740 pothole and 1,106 speed-breaker

annotations, averaging about 1.8 annotations per image. Additional augmentations CLAHE, motion blur, grayscale conversion, and brightness shift were later applied programmatically during training using Albumentations to simulate diverse conditions and improve generalization.

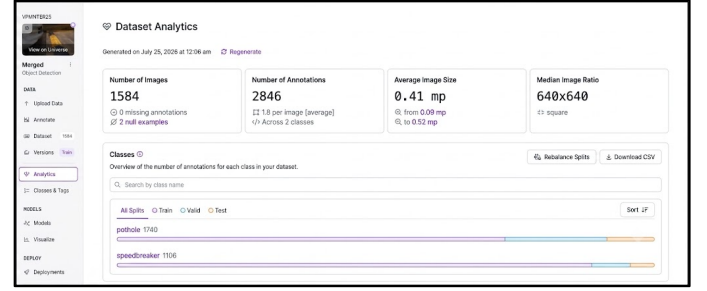


Fig. 5. Class distribution of the final merged dataset in Roboflow.

Overall, this carefully prepared and cleaned dataset provides a solid foundation for training a YOLOv11 detector capable of recognizing potholes and speed breakers from real-time video, ensuring quality, consistency, and variability for the safety-critical domain of autonomous navigation.

IV. METHODOLOGY

A. Problem Formulation

Pothole and speed-breaker detection falls under computer vision, where the objective is to identify and locate road anomalies within image or video frames. The problem is inherently spatial, requiring both classification (what the object is) and localization (where it is). Three approaches are relevant: classification assigns a single label to the whole image but cannot localize; object detection identifies objects and their bounding boxes, suitable for multiple anomalies per frame; and segmentation produces pixel-wise classes but is computationally heavier and less suited to real-time embedded use. Given the need for real-time, localized detection with reasonable computational cost, object detection was selected.

B. Choice of Model Architecture

The YOLO family has become the standard for real-time object detection due to its speed–accuracy trade-off. Unlike two-stage detectors, YOLO treats detection as a single regression problem, predicting bounding boxes and class probabilities directly in one evaluation. YOLOv1–v3 introduced and improved detection speed and accuracy; YOLOv4–v5 focused on better backbones and training tricks; YOLOv8 added native segmentation, classification, and tracking; and YOLOv11 emphasizes speed, low latency, and modularity for edge deployment. YOLOv11 was chosen for its high frame rate, TensorRT compatibility, simplified training and deployment workflows, and competitive accuracy on small datasets with augmentation.

C. Data Preprocessing and Augmentation

Before training, the dataset underwent preprocessing and augmentation to improve generalization and robustness, which is important given its limited size. All images were resized to 640×640 pixels and pixel values were normalized to [0, 1] internally for faster convergence. The Ultralytics framework

applies powerful augmentations in real time using Albumentations and OpenCV-based methods, providing diverse inputs without increasing dataset size. Key augmentations include:

HSV augmentation: randomly alters hue, saturation, and value to simulate lighting and weather variability, encouraging colour-invariant features.

Mosaic augmentation: combines four images into one, exposing the model to more object scales and placements per pass and boosting small-dataset performance.

MixUp augmentation: blends two images and their labels, encouraging soft-label learning and robustness to label noise.

Random affine transformations: scaling, translation, rotation, and shear to emulate varied camera angles and road scenarios.

Horizontal flipping: simulates left–right camera perspectives, beneficial for symmetric classes such as speed breakers.

These augmentations are applied probabilistically per batch and are critical for detecting anomalies across environments ranging from well-lit highways to poorly maintained urban roads. For inference, images undergo only resizing and normalization to preserve real-world inputs.

D. Evaluation Metrics

Model performance was evaluated using standard object-detection metrics. Mean average precision (mAP) measures precision and recall across IoU thresholds (typically @0.5 or 0.5:0.95); higher mAP indicates better accuracy and localization. Precision is the ratio of true positives to all detections, so high precision means fewer false positives. Recall is the ratio of true positives to all ground-truth objects, so high recall means fewer missed detections. Intersection over Union (IoU) measures the overlap between predicted and ground-truth boxes, and a threshold (e.g., 0.5) classifies detections as correct.

V. IMPLEMENTATION

A. System Architecture Overview

The system enables real-time detection of road-surface anomalies using the Ultralytics YOLOv11 Large (YOLOv11l) model. The architecture is modular, allowing efficient training, inference, and result visualization, and is optimized for deployment across platforms including edge devices and cloud services such as Google Colab. Key architectural improvements over previous versions include the C3k2 (Cross-Stage Partial with kernel size 2) block, SPPF (Spatial Pyramid Pooling–Fast), and C2PSA (Convolutional block with Parallel Spatial Attention) components, which enhance feature extraction and support efficient inference on hardware with limited resources (Khanam and Hussain, 2024).

B. Hardware and Software Requirements

Development and testing were performed on Google Colab, so local hardware was not a limiting factor. For optimal real-time performance, a GPU such as the Tesla T4, P100, or V100 with at least 12–16 GB of RAM and about 5 GB of storage is recommended. For deployment, edge options include the NVIDIA Jetson Nano or Xavier with TensorRT support, a USB or streaming camera input, and sufficient power for a GPU-

enabled edge device. The software stack uses Python 3.10+, the Ultralytics library for training and inference, OpenCV for video handling and frame annotation, and Matplotlib for visualizing metrics. Model files include the pre-trained yolo11l.pt base weights and the fine-tuned best.pt weights, with the dataset stored in YOLO format and organized into train, validation, and test folders with a data.yaml configuration.

C. Model Selection and Configuration

After evaluating several state-of-the-art architectures, YOLOv11 Large was selected for its optimal balance of accuracy and inference speed. It achieves high mAP across IoU thresholds, which benefits the detection of small, irregular objects such as potholes and speed breakers; it preserves real-time inference without significant accuracy loss, making it suitable for latency-sensitive settings such as AVs; and, as an Ultralytics model, it offers a user-friendly training interface, Colab integration, and built-in export, quantization, and acceleration support. Bounding-box detection was used rather than segmentation to simplify output and reduce computational load. Prior to finalizing YOLOv11, YOLOv8 was used as a baseline; it delivered promising results but YOLOv11l ultimately offered superior inference speed and accuracy under real-time video scenarios, while the YOLOv8 experiments informed the training strategy.

D. Model Training

The YOLOv11l model was fine-tuned on the custom dataset of potholes and speed breakers using Google Colab with GPU acceleration. Training ran for 100 epochs on a Tesla T4 GPU with the Ultralytics framework, starting from the yolo11l.pt weights pre-trained on COCO. On completion, YOLOv11 organizes outputs in a project directory containing key artifacts: best.pt (the checkpoint with the best validation mAP, used for inference and deployment), last.pt (final-epoch weights, useful for retraining), results.png (a visual summary of training metrics), and additional plots such as the confusion matrix, F1 curve, and precision–recall curves.

The following code was used to train the model:

```
from ultralytics import YOLO

# Load the YOLOv11 model
model = YOLO("yolo11l.pt")

# Train the model
model.train(
    data="/content/dataset/data.yaml", # Update with dataset path
    epochs=100,
    imgsz=640,
    batch=16,
    project="YOLOv11-Training",
    name="pothole-speedbump-detect"
)
```

E. Video Output

For video inference, the trained YOLOv11l model runs on a video input to detect road hazards. Using OpenCV, the video is opened and its width, height, and frame rate are extracted to configure the output writer with matching specifications. Each frame is processed with a confidence threshold of 0.4, and the plot() method annotates it with detected bounding boxes and

class labels; annotated frames are written sequentially to an output video. This enables automated, frame-by-frame detection without a GUI display, making it ideal for headless environments such as Colab.

This script performs real-time inference using the trained YOLOv11 Large model (best.pt) on a video input (bump2.mp4) to detect road hazards like potholes, broken roads, and unmarked speed bumps.

```
# Load the trained model
model = YOLO("content/best.pt") # Use correct path to model

# path to video file
cap = cv2.VideoCapture("bump2.mp4")

# Get video properties
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS) or 30 # Default to 30 if not detected

# Define the codec and create VideoWriter object
# saves outputted video to directory with annotated boxes after detection
out = cv2.VideoWriter('output.mp4',
                      cv2.VideoWriter_fourcc('mp4v'),
                      fps,
                      (frame_width, frame_height))

# Loop through frames
while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Run YOLOv11 prediction
    results = model.predict(source=frame, imgsz=640, conf=0.4, stream=True)

    # Annotate and write to output
    for r in results:
        annotated_frame = r.plot() # Draw masks and boxes
        out.write(annotated_frame)

# Release everything
cap.release()
out.release()
```

VI. RESULTS AND ANALYSIS

This section evaluates the trained YOLOv11 model in terms of detection accuracy, speed, and real-time performance, using standard metrics, visual results on test images and video frames, and hardware benchmarking. Results were obtained after training for 100 epochs with a 640×640 input size and a batch size of 16, and are analysed through the confusion matrix, confidence-precision and recall curves, and bounding-box distribution.

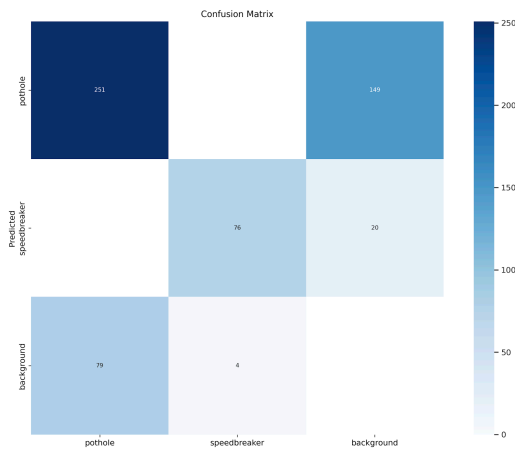


Fig. 6. Confusion matrix of the trained YOLOv11 model.

The confusion matrix clarifies the model's classification behaviour across the pothole, speedbreaker, and background classes. The model correctly predicted 251 pothole instances but misclassified 149 potholes as background, indicating missed detections. Similarly, 76 speed-breaker instances were correctly detected, while 20 were labelled as background and 79 pothole instances were predicted as speed breakers. These errors highlight difficulty in differentiating faded or small anomalies and in distinguishing anomalies from background under poor visibility, yet the model retains strong classification for clear, well-annotated samples.

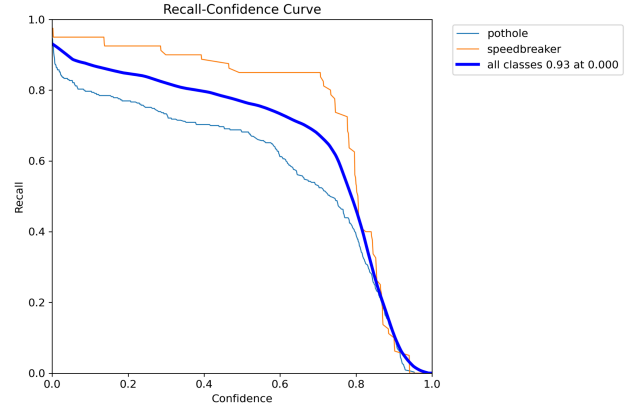


Fig. 7. Recall versus confidence curve.

The recall-confidence curve shows how recall varies with the confidence threshold. The model maintains relatively high recall (>0.85) for both classes across a wide range of confidence scores, indicating that it detects most true positives at low-to-moderate thresholds. As the threshold increases, recall declines, as expected when stricter thresholds filter borderline predictions. The speed-breaker curve remains slightly higher than the pothole curve, suggesting better generalization for that class due to clearer structural features.

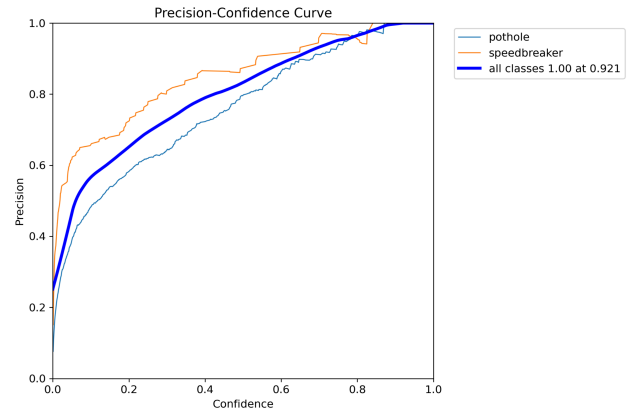


Fig. 8. Precision versus confidence curve.

The precision-confidence curve indicates how confident the model is in its predictions. At low thresholds the model produces more false positives, especially for potholes, lowering precision; as confidence increases, precision rises steadily and peaks near 1.0. Speed breakers show a steeper, more consistent climb owing to their more defined structure. The combined all-classes curve

peaks at a confidence threshold near 0.92, indicating the point of maximum precision without losing many true positives.

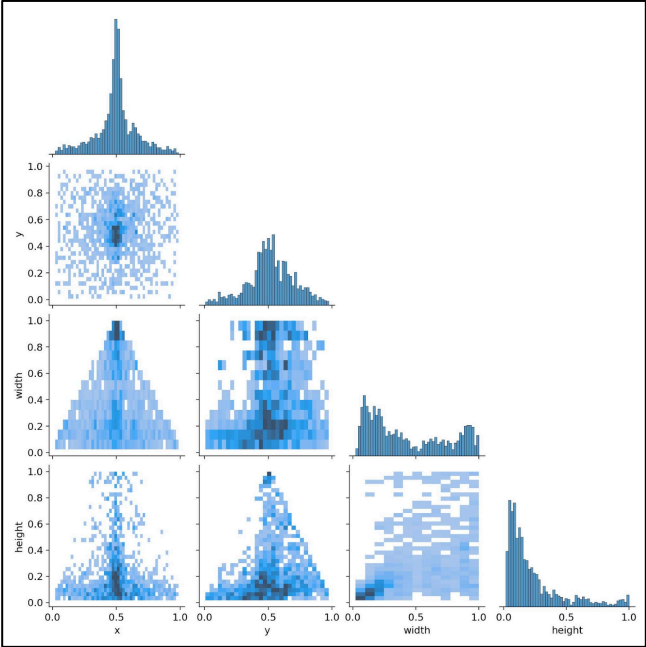


Fig. 9. Correlogram of the normalized bounding-box parameters.

To evaluate bounding-box quality and consistency, a joint distribution heatmap of the normalized box parameters (x, y, width, height) was generated. Boxes concentrate toward the centre of the frame, consistent with dashcam captures where anomalies typically appear. Width and height distributions show that most boxes are relatively small, as expected for potholes and narrow speed breakers, with some spread reflecting scale variance. This confirms that the training data was diverse and realistic in object placement and scale.

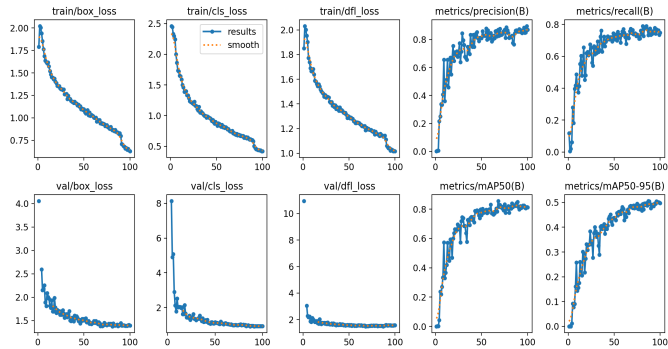


Fig. 10. Training and validation performance over 100 epochs.

The training summary tracks box loss, classification loss, and distribution focal loss (DFL) alongside precision, recall, and mAP over 100 epochs. The bounding-box regression loss steadily decreases from about 2.0 to below 0.75, and the classification loss falls from about 2.5 to below 0.5, confirming improved localization and class discrimination. The DFL loss shows a stable decline. Precision improves to above 0.85 and recall stabilizes near 0.80. The $mAP@0.5$ crosses 0.85, while validation losses that start high (the DFL loss peaking near 11) converge by the 30th–50th epoch, indicating good

generalization without overfitting. The $mAP@0.5:0.95$ rises from nearly 0.0 to just below 0.50, a strong result for a two-class task with subtle features.



Fig. 11. Validation predictions on pothole images.

Validation results show the model’s predictions on unseen images. For potholes, the model detects anomalies in urban streets, rural roads, and water-logged areas, often with confidence above 0.8, and places bounding boxes accurately even under uneven surfaces, varying lighting, and clutter. A few overlapping or lower-confidence detections (around 0.3–0.5) reflect sensitivity to partial occlusion and texture noise.



Fig. 12. Validation predictions on speed-breaker images.

For speed breakers, the model is consistent across striped, painted, and modular plastic variants, with many detections above 0.7–0.8 and boxes that align well with object boundaries, demonstrating generalization across marking styles, angles, and backgrounds. Overall, the loss curves converge in both training and validation, precision and recall improve steadily, and high

mAP@0.5 and mAP@0.5:0.95 confirm reliable detection, establishing a solid foundation for deployment on edge devices or integration into AV perception pipelines.

VII. INFERENCE SPEED AND HARDWARE DEPLOYMENT

To evaluate real-world feasibility, the trained YOLOv11 model was tested for inference speed on both high-performance systems and resource-constrained edge devices, ensuring it meets the latency and performance requirements of real-time road-condition monitoring.

A. Inference Speed

TABLE I. INFERENCE SPEED ACROSS DEVICES (640×640)

Device	Avg. Inference Time	Avg. FPS
CPU	1643.5 ms/frame	0.89
NVIDIA T4 GPU	47.5 ms/frame	30.2

On the CPU, inference time was prohibitively high (about 1.6 s per frame), making it unsuitable for real-time deployment. On the T4 GPU, the model achieved 30.2 FPS, suitable for both image- and video-based real-time applications, although high post-processing time was observed due to extensive bounding-box decoding and visualization (preprocessing and post-processing times were 13.7 ms and 424.8 ms on the T4, and 4.9 ms and 4.3 ms on the CPU, respectively).

B. Hardware Compatibility Assessment

Hardware feasibility was analysed across common embedded and GPU devices. The Jetson Nano has limited compatibility due to lower memory and compute; it requires INT8 quantization for acceptable speed and is not suitable for real-time use without TensorRT optimization. The Jetson Xavier is fully supported and delivers moderate-to-high performance, making it suitable for compact, efficient edge deployment. The Raspberry Pi 4 is not recommended, as the 49 MB model is too large and power-intensive for practical deployment. NVIDIA T4, V100, and A100 GPUs are fully compatible and optimized for high-speed batch inference and video-stream processing, making them suitable for development and server-side production.

VIII. MODEL OUTPUT AND DISCUSSION

A. Image Inference

During inference on individual images, the YOLOv11 model detects both potholes and unmarked speed bumps accurately, identifying multiple hazards within a single frame even when they vary in size or orientation or are partially occluded. It consistently draws precise bounding boxes with high confidence, and detections are highly reliable in well-lit conditions and on standard asphalt surfaces with minimal false positives, highlighting its potential for real-time visual-inspection systems and AV pipelines. Sample outputs are shown below.



Fig. 13. Detection result on a test image.

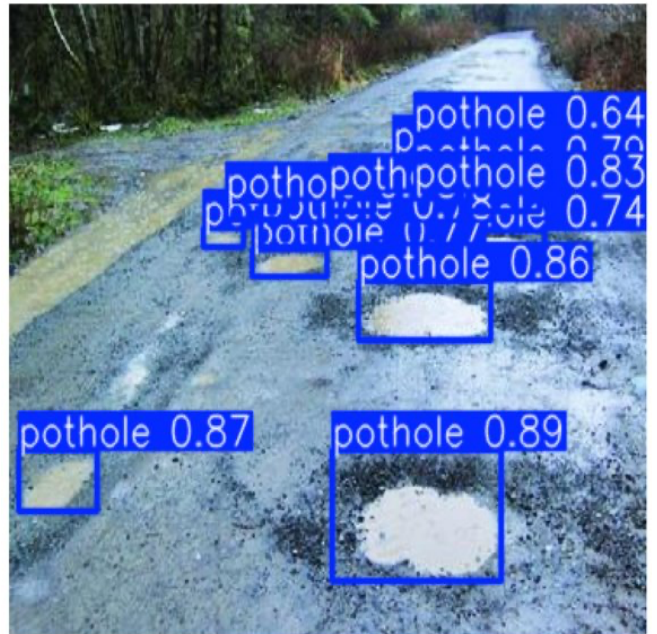


Fig. 14. Detection result on a wet road with multiple potholes.

B. Video Inference

The trained model was evaluated on real-world driving video dashcam footage of Main St. West near McMaster University—to simulate deployment in an AV scenario. Using OpenCV, the video was processed frame by frame, with each frame passed to the model; detected potholes and speed bumps were visualized with bounding boxes, and annotated frames were compiled into a final output video. The model performed particularly well at detecting multiple potholes within the same frame on heavily broken road, identifying potholes of varying sizes and shapes—even faded, cracked, or clustered ones—with tightly aligned boxes. Inference speed was sufficient for near real-time processing with GPU acceleration.



Fig. 15. Detection on live dashcam footage (Main St. W., McMaster University).

Some limitations were observed under challenging conditions. Strong shadows, sun glare, or reflections on the car hood occasionally led to false positives or missed detections. Notably, the model sometimes predicted manhole covers as potholes, which is understandable since manhole covers resemble a divot in the road. Despite this, the model maintained stable performance across most typical road scenes and was especially effective in urban environments where defects are frequent and clearly visible. Overall, the video results demonstrate reliable operation in dynamic, real-world scenarios and underscore the practical feasibility of deploying the model as a core module for road-condition assessment and hazard avoidance.

IX. LIMITATIONS AND FUTURE WORK

A. Limitations

Despite promising results, the system faces limitations that affect real-world deployment. Performance drops significantly under extreme weather such as heavy rain, snow, fog, or intense glare, owing to limited training data and insufficient augmentation for these edge cases, and accuracy decreases in low-light or night-time scenarios without specialized preprocessing. Real-time processing of high-resolution video is computationally demanding on low-power or embedded hardware, causing delays that hinder timely decision-making.



Fig. 16. Example of misclassification: a manhole cover detected as a pothole.



Fig. 17. Example of reduced accuracy in low-light conditions.

The training data lacks diversity in rare but critical conditions such as snow-covered or water-filled potholes, and includes misclassifications such as confusing manhole covers with potholes, highlighting the need for more robust and varied labelling. Effective deployment also requires integration with other sensors (e.g., LiDAR, radar) and GPS data, which introduces calibration, synchronization, and compatibility complexity, and hazard mapping with dynamic rerouting demands additional back-end infrastructure.

B. Future Improvements

Dataset expansion with edge cases. Expanding the training data to include night-time, fog, snow, glare, and heavy rainfall scenarios that are common in practice but underrepresented would improve generalization and maintain performance under diverse conditions.

GPS integration for hazard mapping. Integrating GPS with the detection pipeline would geotag hazards in real time, enabling dynamic road-condition maps and automated reporting to municipal authorities or navigation systems, supporting proactive maintenance and improving road safety for all drivers.

Transformer-based backbones. Replacing or augmenting the CNN backbone with transformer-based architectures such as YOLO-World could improve contextual awareness through self-attention, helping the model understand global scene context and detect objects that are partially occluded or poorly illuminated, and reducing misdetections of objects such as trees and buildings.

Integrated lane detection for context-aware filtering. Adding a lane-detection module would restrict detection to drivable regions, reducing false positives such as trees or clouds and ensuring that only relevant in-lane anomalies are flagged—particularly useful for AV applications.

Model light-weighting for edge deployment. Quantization (e.g., FP32 to INT8) reduces model size and computation, while TensorRT optimization accelerates inference on NVIDIA GPUs by streamlining the computation graph, making real-time deployment feasible without sacrificing accuracy.

Aerial and satellite-based detection. For large-scale or rural assessment, integrating drone or satellite imagery offers a broader perspective, enabling identification of damaged infrastructure at regional or national scale to supplement AV data and prioritize repairs.

X. REFERENCES

- [1] R. Khanam and M. Hussain, "YOLOv11: An overview of the key architectural enhancements," arXiv preprint arXiv:2410.17725, 2024.
- [2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788.
- [3] C. Koch and I. Brilakis, "Pothole detection in asphalt pavement images," Adv. Eng. Inform., vol. 25, no. 3, pp. 507–515, 2011.
- [4] Y. Zhang, S. Süssstrunk, and M. Salzmann, "Automated vision system for road condition assessment," Procedia Comput. Sci., vol. 32, pp. 342–349, 2014.
- [5] H. Maeda, Y. Sekimoto, T. Seto, T. Kashiyaama, and H. Omata, "Road damage detection using deep neural networks with images captured through a smartphone," arXiv preprint arXiv:1801.09454, 2018.
- [6] G. Saini, R. Sharma, and P. K. Sharma, "Real-time pothole detection using YOLO," Int. J. Comput. Appl., vol. 176, no. 28, pp. 1–4, 2020.
- [7] A. Kumar, M. Singh, and P. Garg, "Road anomaly detection using SSD and MobileNet for smart cities," in Proc. IEEE SmartTech Conf., 2021, pp. 198–202.
- [8] A. Patil, S. Pawar, and R. K. Kharat, "Pothole detection on Jetson Nano using YOLOv4-Tiny," Int. J. Eng. Res. Technol. (IJERT), vol. 10, no. 5, pp. 245–248, 2021.
- [9] M. Rai, S. Singh, and A. Sharma, "Real-time pothole detection and mapping using YOLOv5 and GPS," in Proc. Int. Conf. Commun. Technol. (ICCT), 2022, pp. 89–94.